Gebze Technical University
Computer Engineering Department
CSE 396 – Computer Engineering Project
2025–2026 Spring

SeeFire

Autonomous Fire Detection Robot with Edge Computing

Project Proposal

Team Members	**Student ID**
Emre Can Tuncer	200104004115
Batuhan Yanar	220104004059
Ahmet Furkan Arslan	220104004044
Alperen Şahin	220104004943
Semih Sarıkoca	220104004038
Yusuf Alperen Çelik	220104004064
Bekir Emre Sarıpınar	220104004039
Halil Buğra Şen	230104004088

Instructor: Dr. Salih Sarp

Contents

1 Introduction and Problem Statement **2**

2 Literature Review and References **3**

2.1 Vision-Based Fire Detection with YOLO 3

2.2 Edge AI for Fire Detection 3

2.3 Indoor Navigation for Mobile Robots 3

3 Hardware Architecture and Data Collection **4**

3.1 Components and Budget 4

3.2 3D Printed Parts 4

3.3 Data Collection 4

4 Software Methodology I: AI and Computer Vision **5**

4.1 Why YOLOv8n 5

4.2 Detection Pipeline 5

4.3 Sensor Fusion 5

5 Software Methodology II: Navigation and Mapping **6**

5.1 How the Robot Moves 6

5.2 State Machine and Fire During Exploration 6

5.3 Mapping and Map Persistence 7

5.4 Dealing with Position Drift 7

5.5 Obstacle Avoidance 7

6 Evaluation and Success Metrics **7**

6.1 Minimum – Must Have 7

6.2 Target – Should Have 8

6.3 Test Plan 8

7 Final Product and Deliverables **8**

8 Team Structure **8**

9 Risk Analysis **10**

1 Introduction and Problem Statement

Fires in enclosed industrial spaces—warehouses, factories, storage units—are a well-known safety problem. These places tend to be large, poorly ventilated, and full of flammable materials. If a fire breaks out at night or over the weekend when nobody is around, it can grow out of control before anyone even notices.

Most buildings rely on ceiling-mounted smoke detectors and heat sensors. They work, but they have obvious drawbacks: fixed coverage, blind spots in large open areas, no way to pinpoint *exactly* where the fire started, and a tendency to produce false alarms. They also cannot give you any visual confirmation—you just get a beeping alarm and have to figure out the rest yourself.

We want to build something better. **SeeFire** is an autonomous wheeled robot that patrols indoor areas and actively looks for fire hazards. It carries a USB camera to spot flames and smoke visually, an infrared temperature sensor to catch heat anomalies, and a gas sensor to pick up smoke particles in the air. All of this processing happens on the robot itself—on a Raspberry Pi 4—without any internet connection or cloud servers. We are going with an **edge computing** approach, meaning the robot decides on its own, locally, which gives us faster reaction times and zero dependency on network infrastructure.

If the robot spots something suspicious, it approaches the area to double-check. If the threat is confirmed, it activates LED lights and a buzzer to alert anyone nearby, and logs the event with its coordinates on a 2D map it has been building while driving around. That map gets saved as a JSON file so the robot can reload it on next startup without re-exploring. Over time, this spatial record of the area acts as a very basic digital twin of the patrolled space.

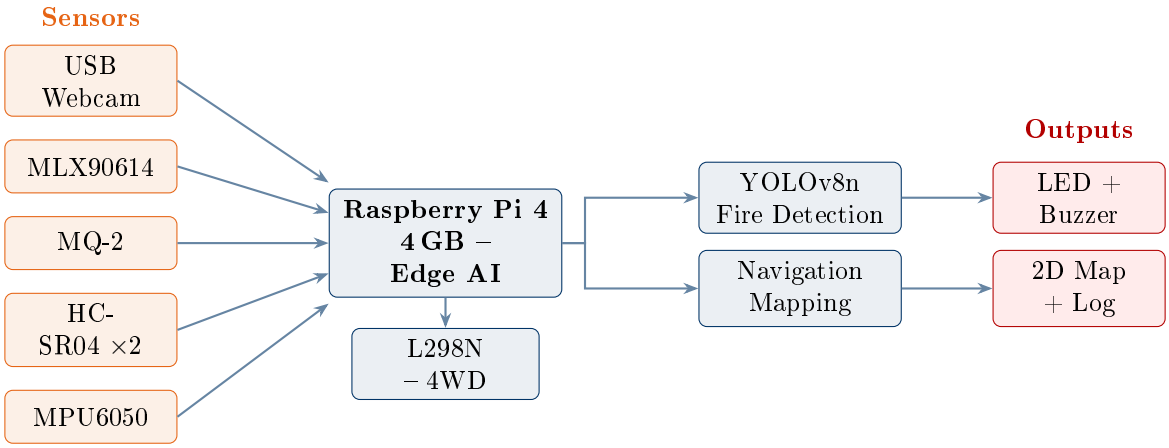

Figure 1: High-level system architecture of SeeFire.

In short, SeeFire combines autonomous navigation, multi-sensor data, and on-device AI into a mobile fire detection unit that is cheap to build, works fully offline, and does not need any human supervision during patrol.

2 Literature Review and References

We looked into recent work on vision-based fire detection, edge AI deployment, and mobile robot navigation to see what has been done and where our project fits in.

2.1 Vision-Based Fire Detection with YOLO

The YOLO family has become a go-to choice for real-time fire and smoke detection. Talaat and ZainEldin [1] applied YOLOv8 to fire detection in smart city scenarios and showed solid accuracy with real-time speed. Following that, several groups tried to push YOLOv8 further for lightweight deployment: Deng et al. [5] added SlimNeck to YOLOv8 and got a 2.7% recall improvement plus 35 extra FPS, while Xu et al. [6] proposed DSS-YOLO, a modified YOLOv8n that focuses on catching small fires in their early stages—the exact scenario we care about.

Li et al. [4] built a fire and smoke detector for factory environments, trained on 5000+ factory images. Their results showed that YOLOv8n gave the best balance between accuracy and speed, hitting about 95.6% mAP@50 after their modifications.

2.2 Edge AI for Fire Detection

Running inference on small boards like Raspberry Pi is no longer just theoretical. Hossain et al. [2] deployed a quantized YOLOv8n on a Raspberry Pi 5 mounted on a drone and got about 89% accuracy in live field tests at roughly 8 FPS. That is encouraging for us since we are using Pi 4, which is slower, but our robot also moves slower than a drone so we can afford a slightly lower frame rate.

The closest work to ours is probably Andriyadi et al. [3], who built a ground robot combining YOLOv8 with an MQ-5 gas sensor. They reported 93% precision and 97% recall. The main difference is that their system sends data over MQTT to an external server, while we keep everything on-device.

2.3 Indoor Navigation for Mobile Robots

For navigating unknown indoor spaces, wall-following is one of the simplest approaches that actually works. More advanced SLAM methods (GMapping, Cartographer) exist in the ROS ecosystem, but they need LiDAR, which would blow our budget. We plan to use ultrasonic-based wall-following with grid-based occupancy mapping and MPU6050 for heading—simpler, but good enough for a structured indoor space.

Terven et al. [9] provide a thorough overview of YOLO architectures, which helped us understand the trade-offs between model size and accuracy when picking YOLOv8n.

3 Hardware Architecture and Data Collection

3.1 Components and Budget

We picked components based on cost, Pi compatibility, and community support. The estimated total budget is approximately **5,100 TL** as of March 2026, based on prices from Robotistan, Direnc.net, and Akakçe. The Raspberry Pi 4 alone makes up about half the cost; the rest—sensors, chassis, power, wiring—is relatively cheap.

We went with **Pi 4 (4 GB) instead of Pi 5** to save about 1,400 TL. Our workload (YOLOv8n at 320 px with INT8 quantization, sensor polling, motor control) fits within Pi 4's specs. Estimated peak RAM usage is around 1.0–1.3 GB, so 4 GB is plenty.

For the camera, we chose a **USB webcam** over the Pi Camera Module V2. A decent USB webcam costs around 200–300 TL compared to ∼900 TL for the Pi Camera, and it connects through a standard USB port so there are no ribbon cable issues. OpenCV reads from USB cameras just as easily, and the image quality is more than enough for YOLOv8n at 320 px input.

Key hardware: Raspberry Pi 4 4 GB, USB webcam, REX 4WD chassis kit with L298N motor driver, MQ-2 smoke sensor, MLX90614 IR temperature sensor, DHT22 for ambient temperature and humidity, MPU6050 IMU for heading, two HC-SR04 ultrasonic sensors for obstacle detection and wall referencing, LiPo 11.1 V battery with 5 V step-down, RGB LEDs and a passive buzzer for alarms.

3.2 3D Printed Parts

We have access to a 3D printer and will print four custom pieces: a camera bracket for the front, sensor mounting brackets, an electronics enclosure to protect the Pi and breadboard, and an LED panel frame for the alarm lights. These are basic shapes—we will design them in FreeCAD or Fusion 360 with help from friends in mechanical engineering.

3.3 Data Collection

All operational data comes from the robot's own sensors in real time. The webcam grabs frames at 15–20 FPS (resized to 320×320 for inference), analog and digital sensor readings come in every 200–500 ms through GPIO and I2C, and the ultrasonics provide distance data for navigation and wall referencing. Everything gets logged to the MicroSD card with timestamps and estimated position.

For training, we will use existing fire/smoke datasets from Roboflow to fine-tune YOLOv8n. We might also capture some of our own test images to improve performance in our specific demo environment.

4 Software Methodology I: AI and Computer Vision

4.1 Why YOLOv8n

We went with **YOLOv8n**—the nano (smallest) variant—because we need something that runs on a Pi with no GPU. On top of that, we will apply **INT8 quantization** to shrink the model and speed things up, with only a small accuracy trade-off. Input resolution is **320×320 px**, enough to spot a flame from a couple of meters away while keeping inference fast.

4.2 Detection Pipeline

Here is how a single detection cycle works:

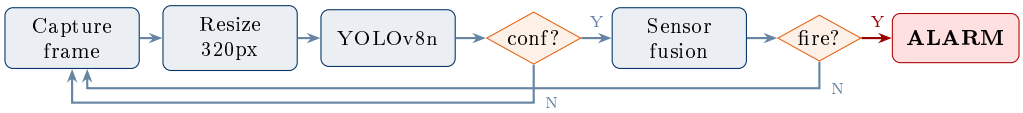

Figure 2: Detection pipeline from frame capture to alarm.

The camera grabs a frame, we resize it, feed it to YOLO, and check the confidence score. If it passes the threshold (somewhere around 0.5–0.7, to be tuned during testing), the detection moves to the sensor fusion stage. There, the system checks whether the other sensors back up the visual detection before making a final call. If the confidence stays below the threshold, we skip the frame and grab the next one.

4.3 Sensor Fusion

The camera alone can be fooled, so we cross-check with other sensors before raising the alarm. If the MQ-2 reads high smoke or gas levels at the same time as a visual detection, our confidence goes up. Same idea with the MLX90614: if it picks up an abnormally hot surface in the direction the robot is facing, that counts as another vote for "real fire." A sudden rise on the DHT22 ambient reading adds yet another signal.

The rule is: the more sensors agree, the more confident we are. The decision engine collects all these inputs and makes a final call. If enough evidence lines up, the alarm triggers. Everything gets saved to local storage either way, so we have a full record of what happened and why.

5 Software Methodology II: Navigation and Mapping

5.1 How the Robot Moves

The robot uses a **wall-following** approach: it keeps a steady distance from the nearest wall using the ultrasonic sensors and traces the perimeter of the room. Not fancy, but reliable, and used in robotics education for years. For a structured indoor space with walls on all sides, it does the job.

5.2 State Machine and Fire During Exploration

The robot goes through four states during operation:

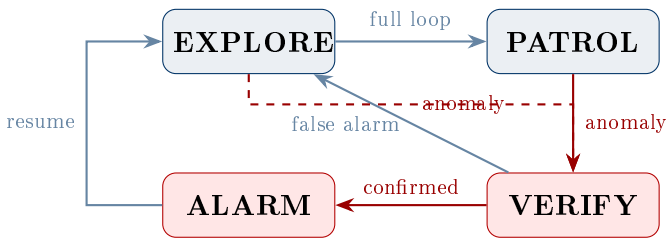

Figure 3: Robot state machine. Dashed line: fire detection during exploration.

1. **EXPLORE** – The robot enters an unknown room, follows the walls, builds a 2D grid map, and picks waypoints along the way. The exploration counts as complete when the robot returns to its starting point (full loop). At that point, the waypoint list is locked and the robot switches to PATROL.
2. **PATROL** – The robot loops through its saved waypoints, scanning with the camera and sensors continuously.
3. **VERIFY** – Something looks off (visual detection, gas spike, or temperature jump). The robot pauses its current route, approaches the suspicious spot, and gathers more data.
4. **ALARM** – Threat confirmed. LEDs go on, buzzer sounds, event is logged with coordinates.

An important detail: **fire detection is always active, including during EXPLORE.** If the robot detects something with high confidence while it is still mapping the room, it goes straight to VERIFY. If the threat is confirmed, it triggers the alarm. After the alarm resets, the robot resumes EXPLORE from where it left off and finishes the map. The exploration is paused, not restarted—the robot picks up from its last known waypoint and continues the loop.

5.3 Mapping and Map Persistence

We use a basic **occupancy grid**: the area is split into cells (roughly 10 cm × 10 cm). As the robot drives around, each cell gets labeled as *free*, *occupied* (wall/obstacle), or *unknown*. If a fire is detected, the corresponding cell gets flagged as a hazard point.

The finished map, along with the waypoint list, is **saved as a JSON file** on the SD card. On next startup, the robot reads this file, loads the map into memory, and goes straight to PATROL without needing to re-explore. This also means the same JSON can be used for visualization—if we want to display the map on a screen or feed it into a monitoring tool, the data is already in a reusable format. The SQLite event log uses the same coordinate system, so hazard locations from the log can be overlaid on the map directly.

5.4 Dealing with Position Drift

Dead reckoning (wheel encoders + MPU6050) will drift over time, which is a known problem. To keep the error in check, we use the HC-SR04 ultrasonic sensors as **wall references**. When the robot is moving along a known wall, it continuously compares the measured wall distance to what the map says the distance should be. If there is a mismatch, it nudges the estimated position to match the wall. This is the simplest form of drift correction and does not need any extra hardware—we are already using the ultrasonics for obstacle avoidance anyway. For our demo area (under 4×4 m), this should keep the map accurate enough.

5.5 Obstacle Avoidance

Two front-facing HC-SR04 sensors constantly measure distance ahead. If something is closer than 20 cm, the robot stops, turns to find a clear path, and continues. This runs at higher priority than wall-following so the robot does not crash into anything.

6 Evaluation and Success Metrics

6.1 Minimum – Must Have

- Robot moves on its own and avoids obstacles.
- Camera detects fire/flame through YOLOv8n.
- LED and buzzer alarms trigger when a fire is spotted.
- At least one supporting sensor (MQ-2 or MLX90614) works alongside the camera.

6.2 Target – Should Have

- Wall-following works and the robot produces a 2D occupancy grid.
- Sensor fusion: camera + gas + IR all feed into the detection decision.
- The robot follows a patrol route through saved waypoints.
- Events are logged with timestamps and map coordinates.
- Map is saved as JSON and reloaded on next startup (no re-explore needed).
- Wall-reference drift correction keeps the map accurate over repeated patrols.
- Detection accuracy above 85% with under 10% false positives in test scenarios.

6.3 Test Plan

We will set up a controlled indoor area (roughly 3 m × 4 m) with walls or barriers. Inside, we place a small controlled flame (alcohol burner in a safe container) and some distractors (warm desk lamp, heat source without flame). The robot needs to patrol the area without collisions, correctly identify the real fire source, ignore the distractors, and log the fire's position on its map.

7 Final Product and Deliverables

On the final demo, we want the robot to perform a complete autonomous run: enter an unfamiliar room, map it by following the walls, switch to patrol mode, spot a fire source with the camera, confirm it with gas and temperature readings, sound the alarm (LEDs + buzzer), and mark the location on its internal map. All sensor data, detections, timestamps, and positions get saved to the SD card. On a second run, the robot should load the saved map and go straight to patrolling without re-exploring.

No Wi-Fi needed. No cloud. No remote control. Just the robot, its sensors, and the Pi 4.

Deliverables: the physical robot (assembled, wired, working), Python source code organized by module, technical documentation for each module, a live demo in our test environment, and a recorded video walkthrough of the project.

8 Team Structure

Eight people, seven modules. Some modules are handled by one person, others by two. Everyone has a primary responsibility and supports at least one other module.

Table 1: Module assignments.

Module	**Key Tasks**
M1: Chassis & Mechanics	4WD chassis assembly, 3D printed parts (camera bracket, sensor mounts, enclosure), physical wiring layout
M2: Motor Control & Power	L298N driver, PWM speed control, differential steering, LiPo battery management, power distribution
M3: Sensor Integration	MQ-2, MLX90614, DHT22, HC-SR04, MPU6050 — wiring, GPIO/I2C drivers, calibration, raw data API
M4: AI & Vision	USB webcam + OpenCV pipeline, YOLOv8n + INT8 optimization, fire/smoke model training, inference benchmarking
M5: Navigation & Mapping	Wall-following, occupancy grid, waypoint generation, patrol loop, obstacle avoidance, wall-reference drift correction, dead-end recovery
M6: Decision Engine	State machine (Explore → Patrol → Verify → Alarm), sensor fusion logic, alarm control (LED + buzzer), explore-pause-resume logic
M7: Data Logging & Output	SQLite logging, timestamped event records, JSON map export/import, SD card management

No dedicated project manager—we run short weekly sprints. The main rule is that every module writes down its API (function signatures, inputs, outputs) in the first week so teams can work in parallel. M1 and M2 are the critical path: nobody tests on real hardware until chassis and motors are ready. Until then, M4 and M5 develop against recorded data and simple simulations.

9 Risk Analysis

Table 2: Risks and how we plan to handle them.

Risk	**What Happens**	**What We Do**
Parts arrive late	Software sits idle, nothing to test on	Develop against pre-recorded data and camera footage. Navigation tested in a Python simulator.
Pi 4 too slow for YOLO	Cannot hit real-time rates	Already using nano + INT8 + 320 px. Worst case: 5–10 FPS, still usable for a slow robot.
Too many false alarms	Robot triggers alarm on non-fire objects	Require 2+ sensors to agree before confirming. Tunable confidence threshold.
Map drifts over time	Dead reckoning errors pile up	Wall-reference correction using HC-SR04. Demo area kept under 4×4 m.
Integration issues	Code from different teams clashes	API contracts in week 1. Weekly check-ins. Shared Git repo.

References

[1] Talaat, F.M. and ZainEldin, H. (2023). An improved fire detection approach based on YOLO-v8 for smart cities. *Neural Computing and Applications*, Springer.

[2] Hossain, M.A. et al. (2024). Real-time fire detection: integrating lightweight deep learning models on drones with edge computing. *Drones*, 8(9), 483.

[3] Andriyadi, A. et al. (2025). Edge AI robot technology application with YOLOv8, MQTT and MQ-5 sensors for real-time fire and smoke detection. *Lecture Notes in Electrical Engineering*, vol. 1358, Springer.

[4] Li, X. et al. (2024). An improved fire and smoke detection method based on YOLOv8n for smart factories. *Sensors*, 24(15), 4786.

[5] Deng, L., Zhou, J., and Liu, Q. (2025). Fire and smoke detection algorithm based on improved YOLOv8. *J. Tsinghua University (Sci. & Tech.)*, 65(4), 681–689.

[6] Xu, Y. et al. (2025). DSS-YOLO: an improved lightweight real-time fire detection model based on YOLOv8. *Scientific Reports*, 15, Nature.

[7] Chen, Z. et al. (2025). FCMI-YOLO: an efficient deep learning-based algorithm for real-time fire detection on edge devices. *Digital Signal Processing*, Elsevier.

[8] Wang, S. and Liu, M. (2024). Multi-scene fire smoke detection algorithm based on YOLO v8. *Proc. ICCNS 2024*, ACM.

[9] Terven, J., Córdova-Esparza, D.M., and Romero-González, J.A. (2023). A comprehensive review of YOLO architectures in computer vision: from YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, 5(4), 1680–1716.